# Experiment 4

# Conditional Statements

## Aim / Objective:

The objective of this experiment is to understand and implement **conditional statements in Java**, which are used to make decisions during program execution based on certain conditions. The experiment aims to:

- Learn the syntax and usage of **if**, **if-else**, **if-else-if ladder**, **nested if**, and **switch-case** statements.
- Develop the ability to apply decision-making constructs to solve real-world programming problems.
- Understand the flow of control in a program based on different logical conditions.
- Enhance debugging skills by observing how different branches of code execute based on inputs.

## .Problem Statement:

1. **Program to check if a number is even or odd**
2. **Program to find the largest of three numbers** **Theory / Concept:**

Conditional statements in Java allow a program to execute different blocks of code based on whether a specified condition evaluates to true or false. These statements are essential in controlling the flow of execution and making decisions during runtime. Java provides several types of conditional statements

**1. if Statement**

The simplest form of a decision-making statement. It executes a block of code only if the condition is true.

```
if (condition) {
   // Code to execute if condition is true }
```

**2. if-else Statement**

Allows the execution of one block of code if the condition is true, and another block if the condition is false.

```
if (condition) {
   // Code if condition is true
} else {
   // Code if condition is false
```

```
}
```

**3. if-else-if Ladder**

Used when multiple conditions need to be checked. It executes the block of the first true condition.

```
if (condition1) {
   // Code if condition1 is true
} else if (condition2) {
   // Code if condition2 is true
} else {
   // Code if none of the above conditions are true }
```

**4. Nested if Statement**

An if statement inside another if or else block. Used for multiple level decision-making.

```
if   (condition1)   {
if (condition2) {
       // Code if both conditions are true
   }
}
```

**5. switch-case Statement**

Best used when a variable needs to be compared with multiple values. It is more readable than using multiple if-else statements.

```
switch (variable)
{     case value1:
// Code block
break;     case
value2:        //
Code block
break;     default:
       // Default block
}
```

**Importance of Conditional Statements** •

Enables decision-making in a program.

- Improves code flexibility and logic.
- Essential for real-world scenarios like login systems, grading systems, etc.

These statements form the basis of control flow in Java and are fundamental to writing logical, interactive, and user-friendly programs.

## Algorithm :

**Program to check if a number is even or odd**

1. Start
2. Input a number
3. Check the remainder when the number is divided by 2 (`number % 2`)
4. If the remainder is 0, the number is **even**
5. Otherwise, the number is **odd**
6. Display the result
7. Stop

**Program to find the largest of three numbers**

1. Start
2. Input three numbers: a, b, and c
3. Compare a with b and c
    a. If a is greater than or equal to both b and c, then a is the largest
    b. Else if b is greater than or equal to both a and c, then b is the largest
    c. Else c is the largest
4. Display the largest number
5. Stop

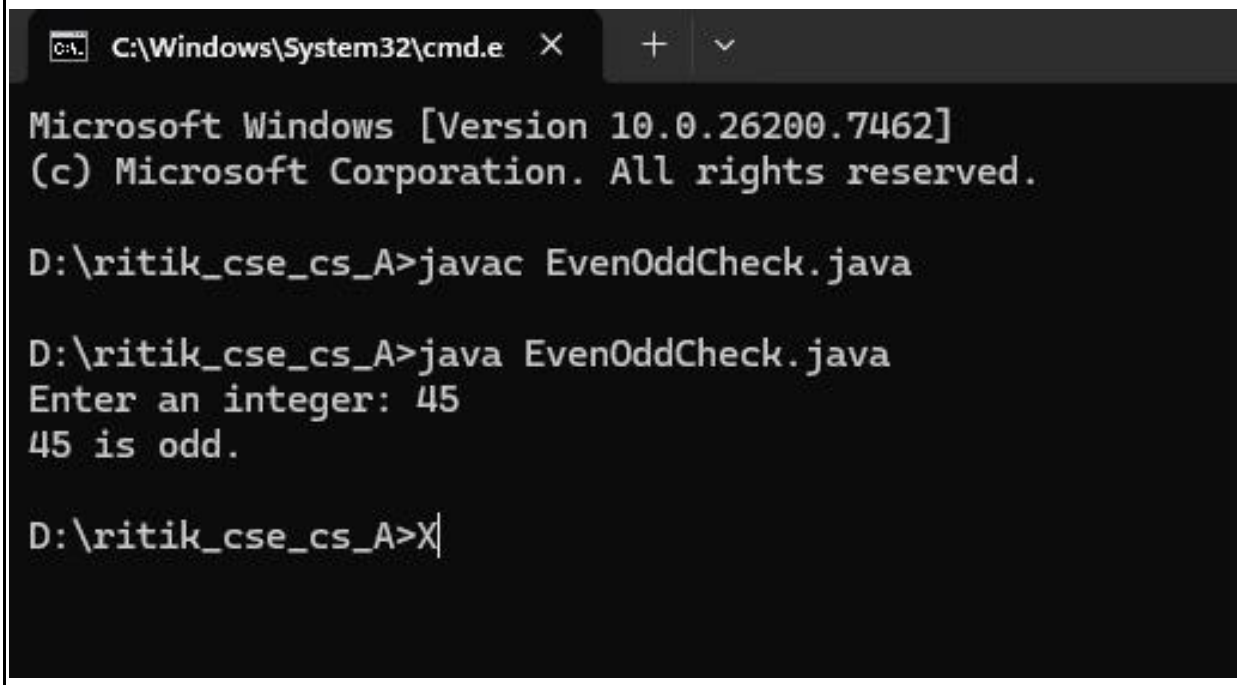
## Program code:

**1. Program to check if a number is even or odd**

```
import java.util.Scanner;
public class EvenOddCheck
{
   public static void main(String[] args)
 {
       Scanner input = new Scanner(System.in);

       System.out.print("Enter an integer: ");
int number = input.nextInt();        if
(number % 2 == 0) {
           System.out.println(number + " is even.");
       }  else
{
           System.out.println(number + " is odd.");
       }
       input.close();
   }
}
```

**Output:**



**2. Program to find the largest of three numbers**

```
import java.util.Scanner;

public class LargestNumber {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in);

      System.out.print("Enter first number: ");
int a = input.nextInt();

      System.out.print("Enter second number: ");
int b = input.nextInt();

      System.out.print("Enter third number: ");
int c = input.nextInt();

      int largest;

      if (a >= b && a >= c) {
         largest = a;
      } else if (b >= a && b >= c) {
         largest = b;
      } else {
         largest = c;
      }

      System.out.println("The largest number is: " + largest);

      input.close();
   }
}
```

**Output**:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

D:\ritik_cse_cs_A>javac LargestNumber.java

D:\ritik_cse_cs_A>java LargestNumber.java
Enter first number: 45
Enter second number: 67
Enter third number: 46
The largest number is: 67

D:\ritik_cse_cs_A>
```

## Compilation and Execution Steps:

Here are the standard steps to **compile and run** a Java program using the command line or terminal:

### Step 1: Write the Java Program

- Create a new text file using any editor (e.g., Notepad, VS Code, or any IDE).
- Save the file with a `.java` extension and ensure the **file name matches the public class name**.

### Step 2: Open Terminal / Command Prompt

- **Windows**: Press `Win + R`, type `cmd`, and press `Enter`
- **Mac/Linux**: Open the Terminal

## Step 3: Navigate to the File Location

- Use the `cd` command to change the directory to where your Java file is saved.
  `cd path\to\your\java\file`

## Step 4: Compile the Java Program

Use the `javac` command (Java compiler) to compile your `.java` file: `javac`

`Example.java`

**Learning Outcomes:**

Students will able to

LO1 Understand Control Statements in java
LO2 Understands the usage of **if**, **if-else**, **if-else-if ladder**, **nested if**, and **switch-case** statements.
LO3 Understands about flow of control in a program based on different logical conditions

**Course Outcome:**

CO3: Apply conditional branching and looping statements.

**Practice Problems:**

1. **Determine if a given year is a leap year.**
2. **Create a program to calculate income tax based on the user's income: Income up to ₹2.5L: No tax**
   **₹2.5L to ₹5L: 5%**
   **₹5L to ₹10L: 20%**
   **Above ₹10L: 30%**

**Code:**

| |
|---|
| **Determine if a given year is a leap year.** |

```
import java.util.Scanner;

public class LeapYearCheck {
   public static void main(String[] args) {
      Scanner sc = new
Scanner(System.in);
System.out.print("Enter a year: ");        int
year = sc.nextInt();

      if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
System.out.println(year + " is a Leap Year.");
      } else {
         System.out.println(year + " is Not a Leap Year.");
      }
   }
}
```

**Create a program to calculate income tax based on the user's income:**

**Income up to ₹2.5L: No tax**

**₹2.5L to ₹5L: 5%**
**₹5L to ₹10L: 20%**
**Above ₹10L: 30%**

```
import java.util.Scanner;

public class IncomeTaxCalculator {
public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter your annual income (in ₹): ");
double income = sc.nextDouble();
```

```
        double tax = 0;

        if (income <= 250000) {
tax = 0;
        } else if (income <= 500000) {
tax = (income - 250000) * 0.05;         }
else if (income <= 1000000) {
            tax = (250000 * 0.05) + (income - 500000) * 0.20;
        } else {
            tax = (250000 * 0.05) + (500000 * 0.20) + (income - 1000000) * 0.30;
        }

        System.out.println("Tax to be paid: ₹" + tax);
    }
}
```

## Conclusion:

In this experiment, we successfully explored and implemented various **conditional statements in Java**, such as if, if-else, if-else-if, and switch-case. These statements enable decision-making in programs based on logical conditions.

Through hands-on coding of examples like checking whether a number is even or odd and finding the largest of three numbers, we understood how control flows based on conditions. This experiment helped strengthen our grasp of how Java evaluates expressions and chooses appropriate code blocks to execute.

## Viva Questions:

1. What is the difference between `if-else` and `switch` statements in Java?
2. Can a `switch` statement handle `String` values in Java? If yes, from which version?
3. What will happen if we forget the `break` statement in a `switch-case` block?
4. How does nested `if` differ from the `if-else-if` ladder in Java?

## For Faculty Use only:

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [ 40%] | Attendance / Learning Attitude [20%] | Total |
|---|---|---|---|---|
| Marks Obtained | | | | |